

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки»

Выполнили:
студенты групп 24ВВВЗ

Масюк А. Р.
Мамонтов Н. П.

Приняли:
Юрова О. В.
Деев М. В.

Пенза 2025

Цель работы

Освоить на практике методы работы с динамическими структурами данных на языке С. Закрепить навыки реализации и использования односвязных списков, приоритетных очередей, стеков и очередей. Изучить принципы динамического распределения памяти и организации данных в виде связанных структур.

Лабораторное задание

Задание:

1. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).
2. * На основе приведенного кода реализуйте структуру данных *Очередь*.
3. * На основе приведенного кода реализуйте структуру данных *Стек*.

Работа программы

```
=== ГЛАВНОЕ МЕНЮ ===
1. Просмотр всего списка
2. Удалить элемент

--- Приоритетная очередь ---
3. Добавить элемент в приоритетную очередь

--- Очередь ---
4. Добавить элемент в очередь
5. Извлечь элемент из очереди
6. Просмотреть очередь

--- Стек ---
7. Добавить элемент в стек
8. Извлечь элемент из стека
9. Просмотреть стек
0. Выход
Выберите действие: 1
Имя - third, Приоритет - 3
Имя - second, Приоритет - 2
Имя - first, Приоритет - 1
```

Реализация приоритетной очереди

```

=== ГЛАВНОЕ МЕНЮ ===
1. Просмотр всего списка
2. Удалить элемент

--- Приоритетная очередь ---
3. Добавить элемент в приоритетную очередь

Очередь
4. Добавить элемент в очередь
5. Извлечь элемент из очереди
6. Просмотреть очередь

--- Стек ---
7. Добавить элемент в стек
8. Извлечь элемент из стека
9. Просмотреть стек
0. Выход
Выберите действие: 6
Содержимое очереди:
1. third
2. second
3. first
4. first1

```

структура данных *Очередь*

```

C:\loavizz\lab3\lab3\x64\Debug\lab3.exe
0. Выход
Выберите действие: 7
Введите название объекта: stck
Элемент добавлен в стек

=== ГЛАВНОЕ МЕНЮ ===
1. Просмотр всего списка
2. Удалить элемент

--- Приоритетная очередь ---
3. Добавить элемент в приоритетную очередь

--- Очередь ---
4. Добавить элемент в очередь
5. Извлечь элемент из очереди
6. Просмотреть очередь

--- Стек ---
7. Добавить элемент в стек
8. Извлечь элемент из стека
9. просмотреть стек
0. Выход
Выберите действие: 9
Содержимое стека:
1. stck
2. third
3. second
4. first
5. first1

```

структура данных *Стек*

Вывод

Освоили на практике принципы работы с динамическими структурами данных в языке С. Закрепили навыки реализации односвязных списков, управления динамической памятью и работы с указателями. Реализовали три основные структуры данных: приоритетную очередь, очередь (FIFO) и стек (LIFO). Написали рабочий код, демонстрирующий операции добавления, удаления и просмотра элементов для каждой из структур. Приобрели практический опыт организации программного кода для работы с динамическими данными.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

struct node
{
    char inf[256];
    int priority;
    struct node* next;
};

struct node* head = NULL, * last = NULL;

struct node* get_struct(void);
struct node* get_struct_with_priority(void);
void spstore(void);
void review(void);
struct node* find(char* name);
void del(char* name);

void priority_enqueue(void);

void enqueue(void);
void dequeue(void);
void view_queue(void);

void push(void);
void pop(void);
void view_stack(void);

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%s", s);
    if (*s == 0)
    {
```

```

        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
    strcpy(p->inf, s);
    p->priority = 0;
    p->next = NULL;

    return p;
}

struct node* get_struct_with_priority(void)
{
    struct node* p = NULL;
    char s[256];
    int pr;

    if ((p = (struct node*)malloc(sizeof(struct node))) == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите название объекта: ");
    scanf("%s", s);
    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    printf("Введите приоритет (целое число): ");
    scanf("%d", &pr);

    strcpy(p->inf, s);
    p->priority = pr;
    p->next = NULL;

    return p;
}

void review(void)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Список пуст\n");
    }
    while (struc)
    {
        printf("Имя - %s", struc->inf);
        if (struc->priority != 0)
            printf(", Приоритет - %d", struc->priority);
        printf("\n");
        struc = struc->next;
    }
    return;
}

void del(char* name)
{
    struct node* struc = head;
    struct node* prev = NULL;
    int flag = 0;

```

```

if (head == NULL)
{
    printf("Список пуст\n");
    return;
}

if (strcmp(name, struc->inf) == 0)
{
    flag = 1;
    head = struc->next;
    if (head == NULL)
        last = NULL;
    free(struc);
    printf("Элемент удален\n");
    return;
}

prev = struc;
struc = struc->next;

while (struc)
{
    if (strcmp(name, struc->inf) == 0)
    {
        flag = 1;
        prev->next = struc->next;
        if (struc == last)
            last = prev;
        free(struc);
        printf("Элемент удален\n");
        return;
    }
    prev = struc;
    struc = struc->next;
}

if (flag == 0)
{
    printf("Элемент не найден\n");
}
}

void priority_enqueue(void)
{
    struct node* p = get_struct_with_priority();
    if (p == NULL) return;

    if (head == NULL || p->priority > head->priority)
    {
        p->next = head;
        head = p;
        if (last == NULL)
        {
            last = p;
        }
        printf("Элемент добавлен в приоритетную очередь\n");
        return;
    }

    struct node* current = head;
    while (current->next != NULL && current->next->priority >= p->priority)
    {
        current = current->next;
    }
}

```

```

p->next = current->next;
current->next = p;

if (p->next == NULL)
{
    last = p;
}

printf("Элемент добавлен в приоритетную очередь\n");
}

```

```

void enqueue(void)
{
    struct node* p = get_struct();
    if (p == NULL) return;

    if (head == NULL)
    {
        head = p;
        last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
    printf("Элемент добавлен в очередь\n");
}

```

```

void dequeue(void)
{
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = head;
    printf("Извлечен элемент: %s\n", head->inf);
    head = head->next;

    if (head == NULL)
    {
        last = NULL;
    }

    free(temp);
}

```

```

void view_queue(void)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    printf("Содержимое очереди:\n");
    int position = 1;
    while (struc)
    {

```

```

        printf("%d. %s\n", position++, struc->inf);
        struc = struc->next;
    }
}

void push(void)
{
    struct node* p = get_struct();
    if (p == NULL) return;

    p->next = head;
    head = p;

    if (last == NULL)
    {
        last = p;
    }

    printf("Элемент добавлен в стек\n");
}

void pop(void)
{
    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    struct node* temp = head;
    printf("Извлечен элемент: %s\n", head->inf);
    head = head->next;

    if (head == NULL)
    {
        last = NULL;
    }

    free(temp);
}

void view_stack(void)
{
    struct node* struc = head;
    if (head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    printf("Содержимое стека:\n");
    int position = 1;
    while (struc)
    {
        printf("%d. %s\n", position++, struc->inf);
        struc = struc->next;
    }
}

int main(void)
{
    setlocale(LC_ALL, "Russian");

    int choice;

```



```

char name[256];

printf("Лабораторная работа №3: Динамические списки\n");

while (1)
{
    printf("\n=== ГЛАВНОЕ МЕНЮ ===\n");
    printf("1. Просмотр всего списка\n");
    printf("2. Удалить элемент\n");
    printf("\n--- Приоритетная очередь ---\n");
    printf("3. Добавить элемент в приоритетную очередь\n");
    printf("\n--- Очередь ---\n");
    printf("4. Добавить элемент в очередь\n");
    printf("5. Извлечь элемент из очереди\n");
    printf("6. Просмотреть очередь\n");
    printf("\n--- Стек ---\n");
    printf("7. Добавить элемент в стек\n");
    printf("8. Извлечь элемент из стека\n");
    printf("9. Просмотреть стек\n");
    printf("0. Выход\n");
    printf("Выберите действие: ");

    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            review();
            break;
        case 2:
            printf("Введите имя для удаления: ");
            scanf("%s", name);
            del(name);
            break;
        case 3:
            priority_enqueue();
            break;
        case 4:
            enqueue();
            break;
        case 5:
            dequeue();
            break;
        case 6:
            view_queue();
            break;
        case 7:
            push();
            break;
        case 8:
            pop();
            break;
        case 9:
            view_stack();
            break;
        case 0:
            printf("Выход из программы...\n");
            return 0;
        default:
            printf("Неверный выбор! Попробуйте снова.\n");
    }
}

return 0;
}

```